

Chapter 1 · Lesson 1 — The Problem (no code yet)

Chapter 1 · Lesson 1 — The Problem (no code yet)

Goal: read this lesson without writing a single line of code, and end with a clear, written-down understanding of what we are about to build and why.

Why this lesson exists before any code

Junior developers open their editor first. Senior developers open a **scratch pad** first. The difference is that seniors know they're going to spend two hours coding the wrong thing if they don't spend ten minutes understanding the *right* thing.

Open a notes file. Or grab a piece of paper. We're going to talk about the app for fifteen minutes before we touch a keyboard, and at the end you should be able to answer a friend asking "*what does the app do?*" in two sentences without hand-waving.

This is the same skill that gets you through interview whiteboarding. The candidate who writes code immediately fails. The candidate who says "*hold on, let me make sure I understand the problem*" and asks two clarifying questions passes.

The pub conversation

Imagine you and seven friends watch the World Snooker Championship every May. Sheffield, the Crucible, two weeks of tense green-baize matches. Someone — let's say it's you — says "*we should make a fantasy league out of this.*" Everyone agrees. Pints are raised.

Now what?

Snooker has a feature that makes fantasy especially cheap to build: **the bracket is published before the tournament starts**. All 32 players are seeded, paired up, and the entire

knockout structure is fixed. Every match in every round is **known in advance** — only the winners are unknown.

That single fact does an enormous amount of architectural work for us. Read it again: *every match in every round is known in advance, only the winners are unknown.*

What follows from it?

- We can ask each of our 8 friends to **fill out the entire bracket up front** — 16 picks for the round of 32, 8 picks for the round of 16, 4 for the quarter-finals, 2 for the semis, 1 for the final. **31 picks per friend, 248 total**, all locked in before the first ball is potted.
- We can list every match by **bracket position**, not by matchId. The matches don't get added or removed during the tournament; only their winner field gets filled in.
- The tournament is **finite**. There's a defined end state — somebody wins the final — at which point every team's score is final too.
- The data is **small**. We're talking ~250 picks, 31 matches, 32 players, 8 teams. That fits in a single JavaScript file. **No database needed.**

That last point is the one that quietly determines the whole stack. If we needed a database, we'd need a backend, which means an API, which means hosting. We don't, so we get to build a static page that anyone can host for free.

Decision dropped out of the problem statement, no opinions required.

The 8 fantasy teams

Real teams from the real app:

Team name	Vibe
Invincibles	"Unstoppable & Unbeatable" ☒
Uncredibles	"Bold predictions, bolder mistakes" ☒
Hopeless	"We tried, we failed, we're charming" ☒
Clueless	"What's a maximum break?" ☒
Break Builders United (BBU)	"Old school grinders" ☒
The Untouchables	"Smooth, calculated, lethal" ☒
Selbies	"Mark Selby fan club, basically" ☒
One Four Sevens	"Maximum vibes only" ☒

Each team is one or two friends sitting at home with a printout of the bracket, picking who they think wins each match.

Notice that the team names are **personality-driven**, not strategy-driven. *Invincibles* isn't a description; it's a brag. *Hopeless* isn't a strategy; it's a shrug. **The data carries the vibe.** That's a pattern: when an app has personality, that personality lives in the data, not the code. The code reads `team.motto`; the data is what makes the motto charming.

The 5 rounds and the 31 matches

The tournament is structured as a single-elimination knockout:

Round 1	(Last 32)	16 matches	Best of 19 frames
Round 2	(Last 16)	8 matches	Best of 25 frames
Quarter-Finals (QF)		4 matches	Best of 25 frames
Semi-Finals (SF)		2 matches	Best of 33 frames
Final		1 match	Best of 35 frames

Add them up: **$16 + 8 + 4 + 2 + 1 = 31$ matches**. The math works because each match eliminates exactly one player, so going from 32 players to 1 champion takes exactly 31 eliminations.

For our app, that means **5 separate “rounds” of data**, each one half the size of the previous. The shape of the data tells you the shape of the UI: probably a tab or a section per round, probably similar-but-not-identical layouts in each.

The scoring rules — this is the heart

There are exactly three rules. Memorize them, because everything else in the codebase is derived from them.

1. **3 points** if your pick wins their match.
2. **1 point** (a “consolation”) if your pick loses their match.
3. **null** — *not zero, not false* — if the match has not yet been played.

Rule 3 is the one beginners get wrong. Let's spell it out.

Imagine it's Day 1 of the tournament. Round 1 is just starting. Out of 16 R1 matches, **zero** are finished. What's *Invincibles'* Round 1 score?

A naive answer says “zero.” But zero is wrong. Zero implies “we computed the score and it came out to zero — like getting all 16 picks wrong.” Zero would put a team that *will* score 30 points indistinguishable from a team that *did* score 0. That’s a bug.

The correct answer is **the score is undefined yet**. It’s not yet a number. It’s `null` — the absence of a number. As matches finish, picks get scored; the score creeps up from null toward its eventual value. **Pending and zero are different states.**

This three-state thinking — *correct, wrong, pending* — runs through every UI screen in the app. The standings table greys out pending columns. The predictions matrix uses three colors (green, red, gray). The pick-list cards have three border colors. They all come back to: **the underlying scoring function returns 3, 1, or null** — never 0 for a pending pick.

If you only take one thing away from this lesson, take this. The most senior thing you can do in a data model is **respect the difference between absent and zero**.

What the app actually has to do (the user stories)

Once the picks are in and the tournament starts, the friends want to be able to:

1. **See who’s leading.** *Standings tab*. A leaderboard with totals and per-round breakdowns.
2. **See match results.** *Matches tab*. Cards per match, marked finished/not-finished, scores visible.
3. **See whose picks were right or wrong.** *Predictions tab*. A matrix comparing all 8 teams’ picks side-by-side, color-coded.
4. **Drill into a player.** *Players tab*. Click any of 32 players, see who they were, who picked them, who picked against them, what they earned for their backers.
5. **See trends.** *Analytics tab*. Bar charts of points per round, accuracy by team, who agreed on what.

Five user stories, five tabs. **The UI is determined by the user stories**, not by what looks cool. If a tab can’t be traced back to a thing one of the eight friends would actually click for, it doesn’t exist.

When you’re prepping for an interview, this is the answer to “*how would you decide what to build first?*” — you start with the user stories, count them, name them, and the architecture falls out.

What the app explicitly does NOT do

Equally important. By saying these out loud you save yourself from over-engineering:

- **No login.** Eight friends, one shared web page. No accounts.
- **No database.** Picks are typed into a JavaScript file by hand. The “admin” updating winners is whoever owns the repo (you).
- **No real-time updates.** When a match ends, the file owner edits winner: 'Wu Yize' and pushes. Friends refresh. That’s the entire ops story.
- **No mobile app.** Just a web page that looks decent on a phone.
- **No multi-tournament support.** This codebase is for *the 2026 Crucible*. Next year’s tournament is a new repo (or a fork).

Each of those “nots” is a feature. Every “no” is a thing you don’t have to build, debug, or maintain. **A senior engineer is as proud of what’s not in the codebase as what is.**

The architecture this implies

We haven’t picked a framework yet, but the problem has already chosen one for us.

- **Single-page app** (one URL, all interaction is client-side).
- **Static deployment** (no backend, no database — just files served from a CDN).
- **Data lives in code** (a `lib/` folder full of TypeScript constants).
- **State is small** (which tab is active, which team or player is selected — that’s basically it).

That description matches **Next.js with the App Router, statically rendered**. Or React + Vite. Or even plain HTML + JS. We’ll pick Next.js in Chapter 2 because it gives us file-based routing, TypeScript out of the box, and zero-config Vercel deployment. But notice we picked it **after** describing the problem, not before. The problem dictated the requirements; the requirements dictated the framework. **Never the other way around.** That’s the difference between a senior and a junior — juniors pick a framework they like and bend the problem to fit; seniors describe the problem and let the framework choose itself.

Two-sentence summary

Try writing this for yourself before reading mine:

An eight-team fantasy snooker league for the 2026 World Championship. Each team locks in 31 picks (16 R1, 8 R2, 4 QF, 2 SF, 1 Final) before the tour-

nement; as matches finish, picks score 3 for a winner and 1 for a loser, and the app shows standings, match results, prediction comparisons, per-player drill-downs, and analytics charts.

If your version says roughly the same thing, you're ready for Lesson 2. If it doesn't, re-read the user stories section.

Vibe prompt you would have used (to verify understanding before coding)

*"I'm about to build a single-page React app for a fantasy snooker league. Eight teams pick winners across five rounds (R1: 16 picks, R2: 8, QF: 4, SF: 2, Final: 1). Scoring: 3 points if the pick wins, 1 if it loses, null while the match is unplayed. No database — bracket and team picks are edited by hand in code. Five tabs: Standings, Matches, Predictions, Players, Analytics. **Don't write any code yet.** Just confirm you understand the problem and ask me anything that's ambiguous."*

The "don't write any code yet" line is the single most useful trick in this lesson. It forces the LLM (and yourself) to surface ambiguities. Try it on every problem: *"summarise the problem back to me, ask three clarifying questions, then I'll tell you to start coding."*

CHECK YOURSELF

Answer these out loud or in writing. If you can't answer one, scroll back up.

1. **Why is null the right value for a pending pick instead of 0?** What goes wrong if you use 0?
2. **Why are there exactly 31 matches in the tournament?** Justify the number from first principles.
3. **The problem statement says "no database." What three follow-on decisions does that single constraint make for us?** (Hint: hosting, auth, who-can-edit.)
4. **Two friends ask to add a 9th fantasy team next year. What's the minimum data change required?** (You don't need to write the code — just say which file/structure you'd touch.)
5. **The five user stories map to five tabs. If the friends asked for a "schedule" tab showing matches in chronological order, would that be a sixth user story or the same as the existing Matches tab?** Argue both sides.

When you've answered these, head to [02-data-modeling.md](#). We'll start translating the problem into TypeScript shapes.